CSE 460 Homework - 1

Exploring Astah and its Features

Problem 1 [20 pts] – Astah Setup and Basic Class Diagram

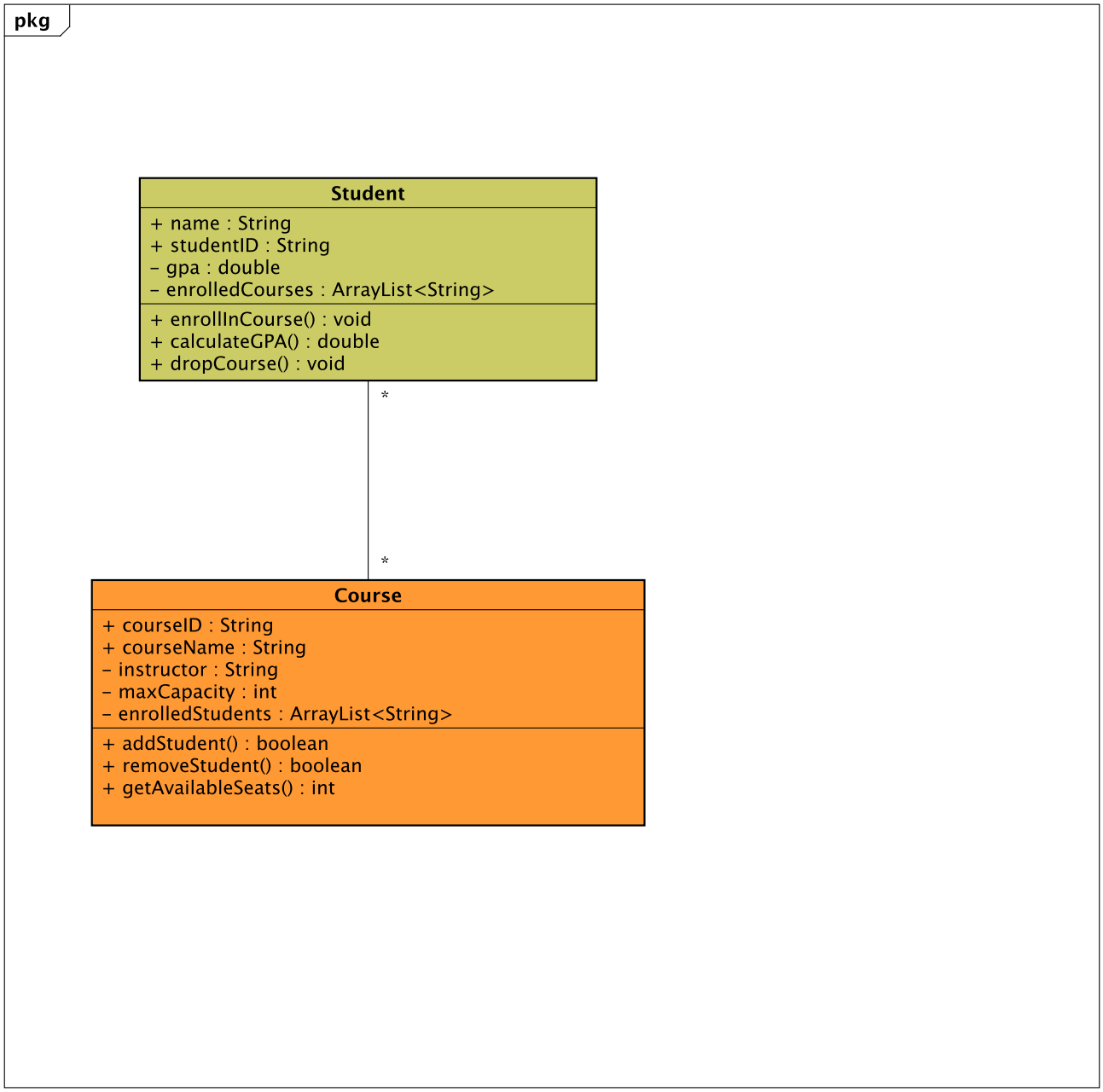

Figure 1: Problem 1 class diagram created in Astah.

Problem 2 [20 pts] – Advanced Class Relationships

A.) Expanded Class Diagram

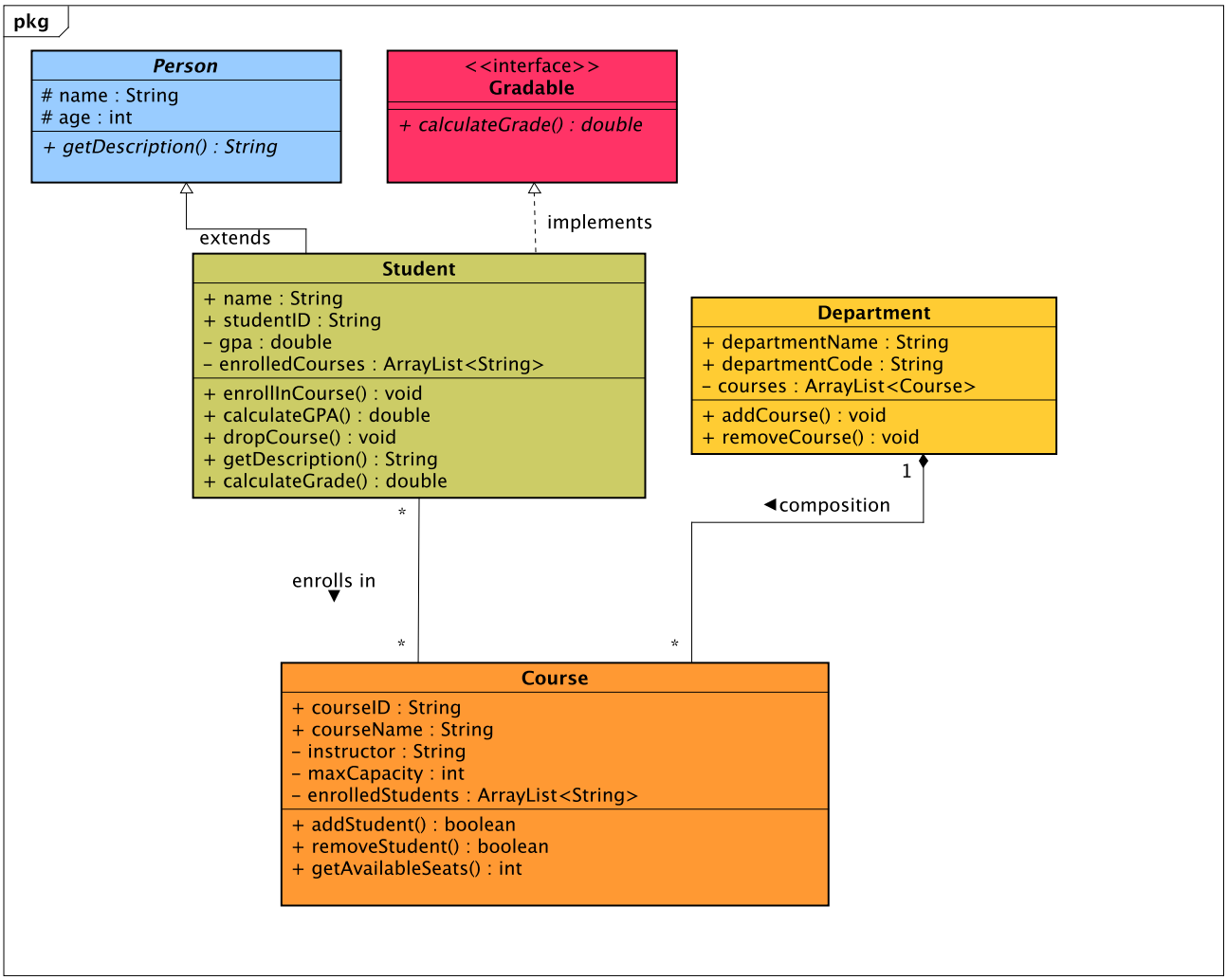

Figure 2: Problem 2 expanded class diagram created in Astah.

B.) Generated Java Code and Mermaid Code

Java code exported from Astah:
https://jumperpc3000.github.io/cse460-software-analysis-design/#assignments

Course.java exported from Astah:
https://github.com/jumperpc3000/cse460-software-analysis-design/blob/main/Assignments/HW1/Exported%20Astah%20java%20/Course.java

Student.java exported from Astah:
https://github.com/jumperpc3000/cse460-software-analysis-design/blob/main/Assignments/HW1/Exported%20Astah%20java%20/Student.java

Person.java exported from Astah:
https://github.com/jumperpc3000/cse460-software-analysis-design/blob/main/Assignments/HW1/Exported%20Astah%20java%20/Person.java

Department.java exported from Astah:
https://github.com/jumperpc3000/cse460-software-analysis-design/blob/main/Assignments/HW1/Exported%20Astah%20java%20/Department.java

Gradable.java exported from Astah:
https://github.com/jumperpc3000/cse460-software-analysis-design/blob/main/Assignments/HW1/Exported%20Astah%20java%20/Gradable.java

Problem 3 [20 pts] – Astah Plugins

1. Plugin Review

For this homework, I installed and used the **Astah Mermaid Plugin**. The main purpose of this plugin is to export Astah class diagrams and sequence diagrams as Mermaid text. This is useful because Mermaid diagrams can be stored as code and rendered directly in tools such as GitHub and Mermaid Live Editor. In my workflow, I first modeled the class diagram in Astah, then used the plugin to export the diagram into Mermaid syntax, and finally verified the output by rendering the Mermaid diagram in GitHub.

The installation process was straightforward. After downloading the plugin `.jar` file, I added it to Astah and restarted the application. The plugin then appeared as enabled in the Astah plugin list. From a usability perspective, the plugin fits naturally into the Astah workflow because the export commands are located under the `Tools` menu. I could export a single diagram by copying the current diagram to the clipboard, or export all supported diagrams to Mermaid files.

The biggest strength of the plugin is that it connects a visual UML modeling tool with a text-based version-control workflow. This makes the diagram easier to share, review, and maintain in GitHub. It also supports important class diagram elements such as classes, attributes, operations, associations, dependencies, realizations, generalizations, parameters, return values, and stereotypes. That support was sufficient for this homework because the design required inheritance, interface realization, association, and composition.

There are also some limitations. The plugin focuses on class diagrams and sequence diagrams, so it is not a general-purpose exporter for every Astah diagram type. For more complex diagrams, the exported Mermaid text may need manual review or minor cleanup to make sure labels, relationship directions, and multiplicities render exactly as intended. Another limitation is that the plugin page states that no technical support is provided, so troubleshooting depends mostly on documentation, experimentation, and careful inspection of the exported code.

Overall, I found the Astah Mermaid Plugin valuable for this assignment. Astah remained the primary modeling environment, while Mermaid gave me a portable text representation of the diagram. This supports good software design practice because it keeps the UML diagram readable for humans while also making the design artifact easier to store and share through GitHub.

2. Proof of Plugin Usage

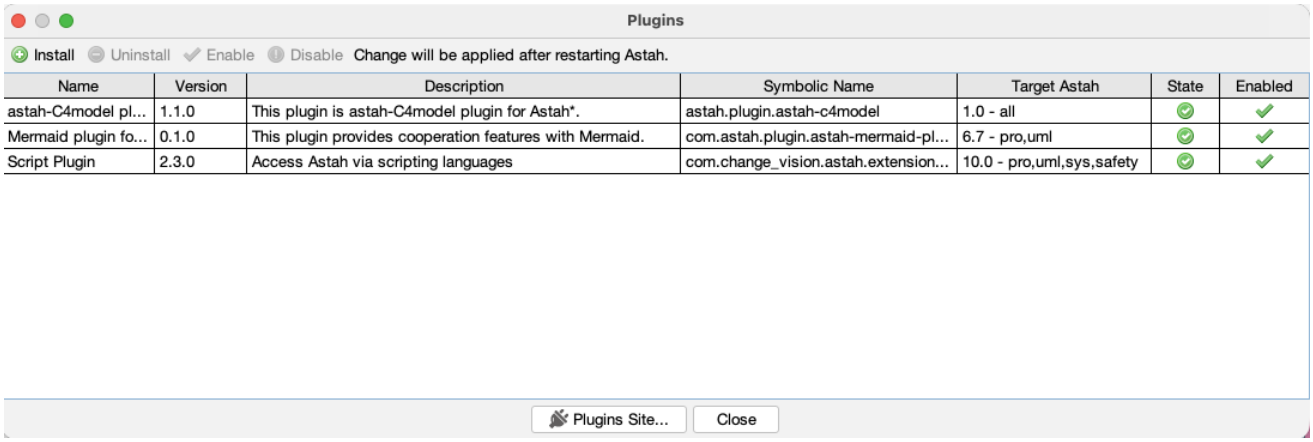

Figure 3: Astah plugin list showing the Mermaid plugin installed and enabled.

Mermaid code exported from Astah before rendering:

```
```mermaid
classDiagram
class Student {
    <<Java Class>>
    +String name
    +String studentID
    -double gpa
    -ArrayList~String~ enrolledCourses
    +enrollInCourse() void
    +calculateGPA() double
    +dropCourse() void
    +getDescription() String
    +calculateGrade() double
}
class Course {
    <<Java Class>>
    +String courseID
    +String courseName
    -String instructor
    -int maxCapacity
    -ArrayList~String~ enrolledStudents
    +addStudent() boolean
    +removeStudent() boolean
    +getAvailableSeats() int
}
class Person {
    <<Java Class>>
    #String name
    #int age
    +getDescription() String*
}
class Gradable {
    <<interface>>
    <<Java Class>>
    +calculateGrade() double*
}
class Department {
    <<Java Class>>
    +String departmentName
    +String departmentCode
    -ArrayList~Course~ courses
    +addCourse() void
    +removeCourse() void
}
Course -- Student
Course "*" -- "*" Student
Course "*" --* "1" Department
Student ..|> Gradable : implements
Person --|> Student : extends
%%Generated by Astah mermaid plugin
```
```

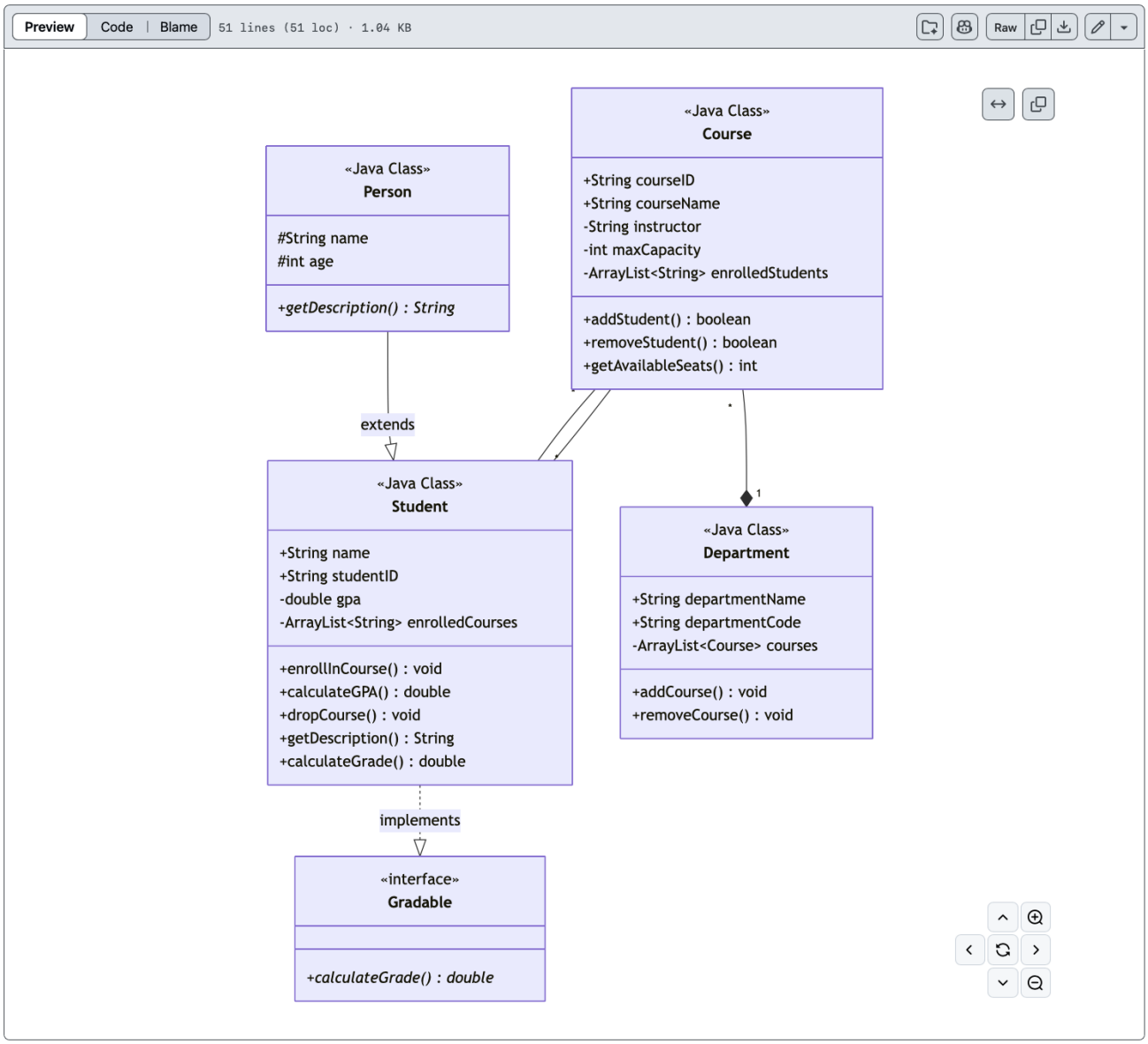

Figure 4: Problem 2 class diagram rendered from Mermaid in GitHub.